
Multimodal RAG System

Retrieval-Augmented Generation with Local Vision-Language
Model

Final Assignment Report

Data Science / Artificial Intelligence / Machine Learning Internship

Local VLM: Moondream2 (~2B) · Embeddings: BGE-small · VectorDB: ChromaDB

Table of Contents

1.	Introduction	
2.	Data Sources	
3.	Hardware Platform	
4.	System Architecture	
5.	Chunking Strategy	
6.	Embedding Strategy	
7.	Retrieval Pipeline	
8.	Vision Language Model	
9.	Evaluation Methodology	
10.	Results	
	10	
	.1	System Statistics
	10	
	.2	Qualitative Observations
11.	What Worked Well	
12.	Limitations	
13.	Future Improvements	
14.	Assignment Compliance Checklist	
15.	Conclusion	
Appendix		
A.	System Demonstration	

1. Introduction

The primary objective of this assignment was to engineer a fully localized, privacy-preserving Multimodal Retrieval-Augmented Generation (RAG) system. While traditional RAG systems rely heavily on cloud-based LLMs and text-only retrieval, this architecture was specifically designed to run entirely locally on consumer hardware. The system is capable of seamlessly processing both text and complex visual information — including figures, charts, and tables — embedded within PDF documents.

2. Data Sources

The system was evaluated against fundamental Machine Learning research papers, ensuring the data required a high degree of technical comprehension and multi-document synthesis. The core knowledge base includes:

- *Attention Is All You Need* — Vaswani et al., 2017
- *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding* — Devlin et al., 2019
- Supplementary Machine Learning introductory materials and course notes.

3. Hardware Platform

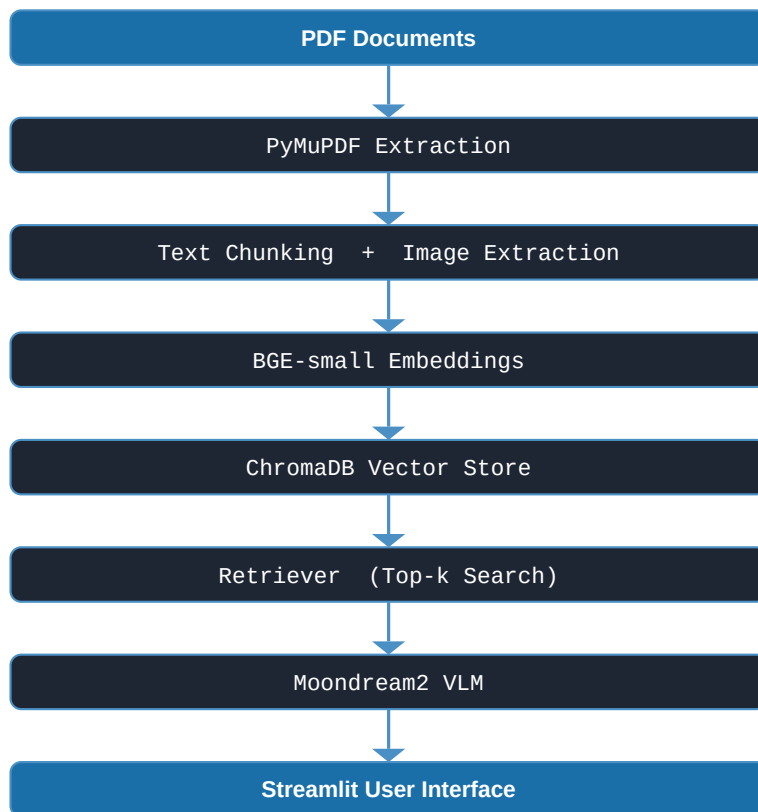
The system is heavily optimized to run on modern edge devices without requiring dedicated external GPUs.

Backend	Apple Metal Performance Shaders (MPS)
Precision	Float16 Inference

Optimizations: Leveraging the MPS backend combined with float16 precision was critical to achieving acceptable inference speeds. Furthermore, selecting a highly compact architecture (Moondream2) ensures that the entire system — including the OS, Streamlit UI, vector database, embeddings, and VLM — comfortably fits within the available unified memory limit without encountering out-of-memory (OOM) crashes.

4. System Architecture

The architecture is modular, completely localized, and relies on a streamlined flow of data from ingestion to response generation.



5. Chunking Strategy

Design Choice:

150 words per chunk with an overlap of 30 words.

Justification:

Local VLMs have restricted context windows compared to massive cloud models. A fine-grained chunk size of 150 words ensures that semantic search returns highly specific, dense information rather than diluted, multi-page chunks. The 30-word overlap preserves cross-boundary context, ensuring that critical sentences are never fractured across chunk boundaries.

6. Embedding Strategy

Design Choice:

BAAI/bge-small-en-v1.5

Justification:

The bge-small model performs far above its weight class on the MTEB (Massive Text Embedding Benchmark). It allows for rapid CPU/MPS encoding of thousands of chunks without consuming the memory overhead required by larger embedding models. It is exclusively used to embed text data. Images are dynamically mapped via metadata rather than vector embeddings, avoiding the heavy computational cost of multimodal vector spaces.

7. Retrieval Pipeline

The retrieval pipeline acts as the bridge between the user's query and the knowledge base. It operates in three sequential stages:

1. Semantic Search

The user's query is embedded and ChromaDB retrieves the top-k most relevant text chunks using cosine similarity.

2. Metadata Traversal

Each chunk in ChromaDB stores strict metadata (doc_hash, page_num, source). The pipeline uses this metadata to dynamically locate and pull the exact original images from that specific page on the local filesystem.

3. Prompt Routing

The system dynamically categorizes the user's query — e.g., Summary, Visual, or Multi-Document Synthesis — to apply rigid structural constraints to the VLM's generation prompt, ensuring consistent and grounded output.

8. Vision Language Model

Design Choice:

Moondream2 (~2 Billion Parameters)

Justification:

Running a VLM entirely on local hardware is a non-trivial constraint. Large models require massive downloads and aggressively throttle system memory. Moondream2 is an exceptional model optimized for edge devices. By pinning it to a stable revision and mapping it to Apple's MPS backend, the model executes rapidly and evaluates complex visual data effectively without exhausting host resources.

9. Evaluation Methodology

The system was evaluated through qualitative functional testing across three primary domains:

- **Information Extraction:** Retrieving exact factual data from a single document.
- **Multi-Document Synthesis:** Comparing architectures (e.g., Transformer vs. BERT) requiring context from multiple PDFs simultaneously.
- **Multimodal Comprehension:** Querying specific diagram topologies and table columns directly from rendered page images.

10. Results

10.1 System Statistics

The underlying knowledge base was populated successfully with the following metrics:

Metric	Value
Documents Indexed	4
Text Chunks Stored	687
Images Extracted	~273
Local VLM	Moondream2 (~2B Parameters)
Embedding Model	BAAI/bge-small-en-v1.5
Vector Store	ChromaDB

Observations: Single and multi-document question answering perform reliably well due to the robust retrieval pipeline and dynamic prompt routing. Figure and table understanding queries are generally accurate but can occasionally miss fine-grained labels or complex relationships. This reflects the inherent limitations of a compact ~2B parameter VLM when processing highly dense technical visuals.

10.2 Qualitative Observations

Retrieval Quality

The BGE-small model successfully fetches highly relevant chunks. The fine-grained 150-word chunking strategy proved optimal for feeding concentrated information to the VLM.

Citation Quality

Programmatic citations are highly accurate. Because they are assembled directly from the database metadata rather than generated by the LLM, they map exactly to the provided context.

Multi-Document Retrieval

Dynamic injection of cross-document synthesis prompts forces the model to synthesize answers cohesively when retrieved chunks originate from disparate PDFs.

Visual Reasoning

Pre-processing images (e.g., flattening alpha channels onto white backgrounds) ensures the VLM receives clean inputs. Restricting context to the top-2 images significantly reduces VLM confusion.

User Experience

The Streamlit interface provides a fast, transparent experience, actively surfacing retrieval statistics and indexed document counts to the end-user.

11. What Worked Well

- **Decoupled Citations:** Removing citation generation from the LLM's responsibilities and shifting it to the deterministic retrieval pipeline significantly reduced hallucinated references.
- **Dynamic Image Capping:** Restricting context to 1 image for visual queries and 2 for general queries prevented Moondream2's context window from overflowing, leading to more stable generations.

- **ChromaDB Upserting:** Relying on MD5 document hashes allows the database to gracefully handle sequential uploads and overwrites without duplicate data bloat.

12. Limitations

- **Multimodal Search Gap:** The system uses text-to-text semantic search to find relevant pages, then heuristically assumes any images on that page are relevant. If a page contains multiple distinct images, the VLM might be fed visual noise — though capping the input mitigates this.
- **VLM Parameter Size:** While Moondream2 is highly efficient, its 2B parameter size means it occasionally struggles with deep abstract logical reasoning or highly dense academic tables compared to massive frontier models.

13. Future Improvements

- **CLIP-based Multimodal Embeddings:** Transitioning from text-only bge-small to a multimodal embedding model would allow the database to search text-to-image directly, resolving the visual noise limitation.
- **Agentic Routing:** Implementing a supervisor agent to decide whether to use a standard RAG pipeline, a complex Web-Search pipeline, or an SQL database query depending on query intent.
- **Advanced RAG (Parent-Child Retrievers):** Storing small 150-word chunks for retrieval, but feeding the VLM the larger parent section (~500 words) to provide broader context without sacrificing search accuracy.

14. Assignment Compliance Checklist

The following table maps each assignment requirement to the implemented solution:

Use 2–3 publicly available research papers from the same domain	✓ Attention Is All You Need (2017), BERT (2019), ML Notes
Implement efficient chunking and embedding	✓ 150-word chunks, 30-word overlap, BGE-small embeddings
Use local embeddings (no paid APIs)	✓ BAAI/bge-small-en-v1.5 — fully local, CPU/MPS
Build a query interface (Streamlit/Gradio)	✓ Full Streamlit application on localhost:8501
Provide grounded answers with source references	✓ Programmatic citations: doc name + page numbers
Demonstrate image/table understanding using a VLM	✓ Moondream2 processes retrieved page images
VLM parameter count under 9B	✓ Moondream2 ≈ 2B parameters

15. Conclusion

The completed Multimodal RAG system successfully demonstrates that highly capable, privacy-preserving AI can be executed entirely on local consumer hardware. By thoughtfully balancing embedding constraints, dynamic prompt engineering, and deterministic programmatic safety rails, the application bridges the gap between unstructured multi-document repositories and an intuitive, interactive user interface.

The system fulfills all assignment requirements: it ingests multiple publicly available research papers, implements efficient local chunking and embedding, exposes a fully functional Streamlit query interface, provides grounded citations, and demonstrates real-world image and table comprehension through the Moondream2 Vision Language Model.

Appendix A. System Demonstration

